

Technische Universität Berlin



Agententechnologien: Grundlagen und Anwendungen

Aufgabe 1 - Projektskizze

Gruppennummer: 19

Gruppenmitglieder:

Gabrijel Bitunjac

Jamie Zimanyi

Diyar Balikci

Datum: 02.Mai.2026

Gliederung:

1. Kurzbeschreibung und Forschungsfrage
2. Hypothese
3. Klassendiagramme
4. Erklärung des Klassendiagramms
5. JSON-Modell
6. Sequenzdiagramm
7. Logging-Konzept
8. Geplante Experimente

1. Kurzbeschreibung der Simulation

In diesem Projekt wird eine Simulation entwickelt, die das Verhalten eines Ameisenhaufens bei der Nahrungssuche nachbildet. Mehrere Ameisenagenten bewegen sich in einer zweidimensionalen Gridwelt, starten aus einem Nest, suchen nach Nahrungsquellen und transportieren gefundene Nahrung zurück.

Die Koordination erfolgt indirekt über Pheromonspuren, die von den Ameisen abgelegt werden und anderen als Orientierung dienen. Dadurch entstehen selbstorganisierte Wege zwischen Nest und Nahrungsquellen.

Die Ameisen handeln reaktiv und treffen Entscheidungen auf Basis lokaler Informationen aus ihrer unmittelbaren Umgebung. Ein zentraler Manager steuert die Simulation, verwaltet die Zeit, führt Aktionen aus und übernimmt die Verwaltung von Energie, Pheromonen und Logging.

Ziel der Simulation ist es, zu untersuchen, wie durch einfache Regeln und lokale Interaktionen ein effizientes Gesamtverhalten entsteht.

2. Hypothese

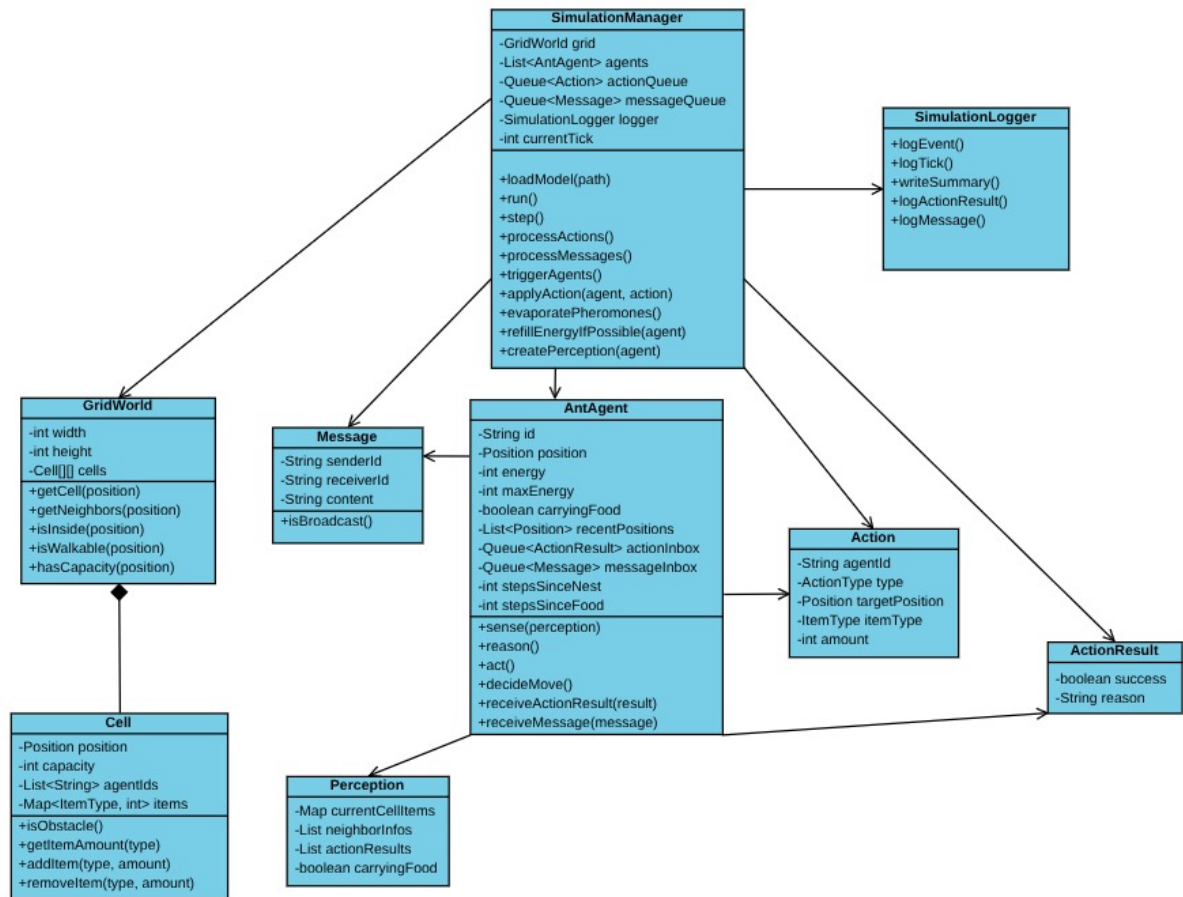
Im Rahmen dieser Arbeit wird untersucht, wie sich die Anzahl der Ameisen auf die Effizienz der Nahrungsbeschaffung auswirkt.

Hypothese:

Die Effizienz der Nahrungsbeschaffung steigt mit zunehmender Anzahl von Ameisen zunächst an, nimmt jedoch ab einer bestimmten Populationsgröße nicht mehr proportional zu. Ursache hierfür sind redundante Wege, verstärktes Pheromonrauschen sowie Konflikte an Engstellen innerhalb der Gridwelt.

Zur Untersuchung dieser Hypothese werden mehrere Simulationen mit variierender Ameisenanzahl durchgeführt. Zusätzlich werden weitere Einflussfaktoren wie die Entfernung der Nahrungsquellen sowie die Pheromonverdunstungsrate berücksichtigt, um deren Auswirkungen auf das Systemverhalten zu analysieren. Zur Bewertung der Effizienz wird insbesondere die insgesamt gesammelte Nahrungsmenge im Verhältnis zurzeit betrachtet.

3. Klassendiagramme



4. Erklärung des Klassendiagramms

Der `SimulationManager` ist die zentrale Kontrollinstanz der Simulation. Er ist selbst kein Agent, sondern verwaltet die Gridwelt, die Agenten, die getaktete Zeit, Energieverbrauch, Konfliktlösung, Pheromonverdunstung und das Logging. Die Agenten verändern die Welt nicht direkt, sondern senden Aktionen an den Manager. Der Manager prüft diese Aktionen und führt sie nur aus, wenn sie gültig sind. Das Aufladen der Energie geschieht implizit und vollautomatisch: Betritt ein Agent ein Nest- oder Nahrungsfeld, setzt der Manager über `refillEnergyIfPossible(agent)` die Energie sofort auf das Maximum. Dies kostet keine extra Zeit und keine Energie.

Die `GridWorld` besteht aus einzelnen Cells. Jede Zelle besitzt eine Kapazität, wobei Kapazität 0 ein Hindernis darstellt. Auf Zellen können Items liegen, zum Beispiel Nahrung, das Nest oder Pheromone. Pheromone werden strikt als Items mit Integerwerten modelliert, sodass die vom Manager gesteuerte Verdunstung über eine einfache Subtraktion erfolgt.

Der `AntAgent` ist ein reaktiver Agent, der vom Manager eine lokale Wahrnehmung (Perception) erhält und anhand von Pheromonspuren entscheidet. Jeder Agent besitzt Energie, Beladungsstatus und optional ein Kurzzeitgedächtnis zur Vermeidung kurzer Zyklen. Um die Vorgabe zu erfüllen, dass Pheromone mit zunehmender Entfernung schwächer abgelegt werden ("decreasing rate"), nutzt der `AntAgent` interne Schrittzähler (`stepsSinceNest`, `stepsSinceFood`). Beim Verlassen von Nest oder Nahrung werden diese auf 0 gesetzt. Pro gelaufenen Schritt erhöht sich der Zähler, und die abzulegende Pheromonmenge wird um einen festen ganzzahligen Wert reduziert.

`Action` beschreibt die vom Agenten gewünschte Handlung. Dabei wird exakt eine Aktion pro Zeittakt durchgeführt. Das Ablegen von Pheromonen kostet keine extra Zeit, weshalb die Klasse `Action` diese Vorgänge vereint. Eine Bewegungsaktion (`targetPosition`) enthält praktischerweise gleichzeitig die Parameter für den kostenlosen Pheromon-Drop (`itemType` und `amount`). Der Manager führt Bewegung und Pheromonablage dann im selben Takt als eine einzige, legale Aktion aus.

`ActionResult` wird vom Manager zurückgegeben und informiert den Agenten im nächsten Sense-Schritt über Erfolg oder Misserfolg. `Message` wird für spätere Erweiterungen vorgesehen, auch wenn in Aufgabenblatt 1 noch keine direkte Agentenkommunikation benötigt wird.

5. JSON-Modell

Zu Beginn eines Experiments wird die Startkonfiguration der Simulation aus einer JSON Datei eingelesen. Dadurch können unterschiedliche Experimente durchgeführt werden, ohne den Programmcode zu verändern. Die Datei enthält unter anderem Informationen zur Gridwelt, zu Nahrungsquellen, Hindernissen, Agenten, Energie, Pheromonen und zur Simulationsdauer.

```
{
  "comment": "Beispiel-Modelldatei: Zeigt ein Grid mit zwei Nahrungsquellen und
  Hindernissen. Demonstriert zudem die Fähigkeit für Warmstarts durch ein initial
  platziertes Pheromon-Item.",
  "seed": 42,
  "simulation": {
    "max_ticks": 2000,
    "energy_cost_per_tick": 1,
    "log_interval": 1,
    "snapshot_interval": 50
  },
  "grid": {
    "width": 18,
    "height": 12,
    "neighborhood": "moore",
    "default_capacity": 999
  },
  "cells": [
    {
      "x": 2,
      "y": 6,
      "capacity": 999,
      "items": [
        { "type": "NEST", "amount": 1 }
      ]
    }
  ]
}
```

```

    ]
  },
  {
    "x": 14,
    "y": 2,
    "capacity": 999,
    "items": [
      { "type": "FOOD", "amount": 40 }
    ]
  },
  {
    "x": 15,
    "y": 9,
    "capacity": 999,
    "items": [
      { "type": "FOOD", "amount": 60 }
    ]
  },
  {
    "x": 3,
    "y": 6,
    "capacity": 999,
    "items": [
      { "type": "PHEROMONE_NEST", "amount": 8 }
    ]
  },
  { "x": 6, "y": 3, "capacity": 0, "items": [] },
  { "x": 6, "y": 4, "capacity": 0, "items": [] },
  { "x": 6, "y": 5, "capacity": 0, "items": [] },

```

```

    { "x": 6, "y": 6, "capacity": 0, "items": [] },
    { "x": 10, "y": 7, "capacity": 0, "items": [] },
    { "x": 11, "y": 7, "capacity": 0, "items": [] },
    { "x": 12, "y": 7, "capacity": 0, "items": [] }
  ],
  "agents": {
    "count": 15,
    "type": "ANT",
    "initial_position": { "x": 2, "y": 6 },
    "max_energy": 120,
    "load_capacity": 1,
    "spawn_mode": "all_at_start"
  },
  "pheromones": {
    "types": ["PHEROMONE_NEST", "PHEROMONE_FOOD"],
    "base_strength": 10,
    "evaporation_amount": 1,
    "min_value": 0
  },
  "policy": {
    "exploration_probability": 0.15,
    "pheromone_weight": 4,
    "memory_length": 4,
    "memory_penalty": 1
  }
}

```


Erklärung:

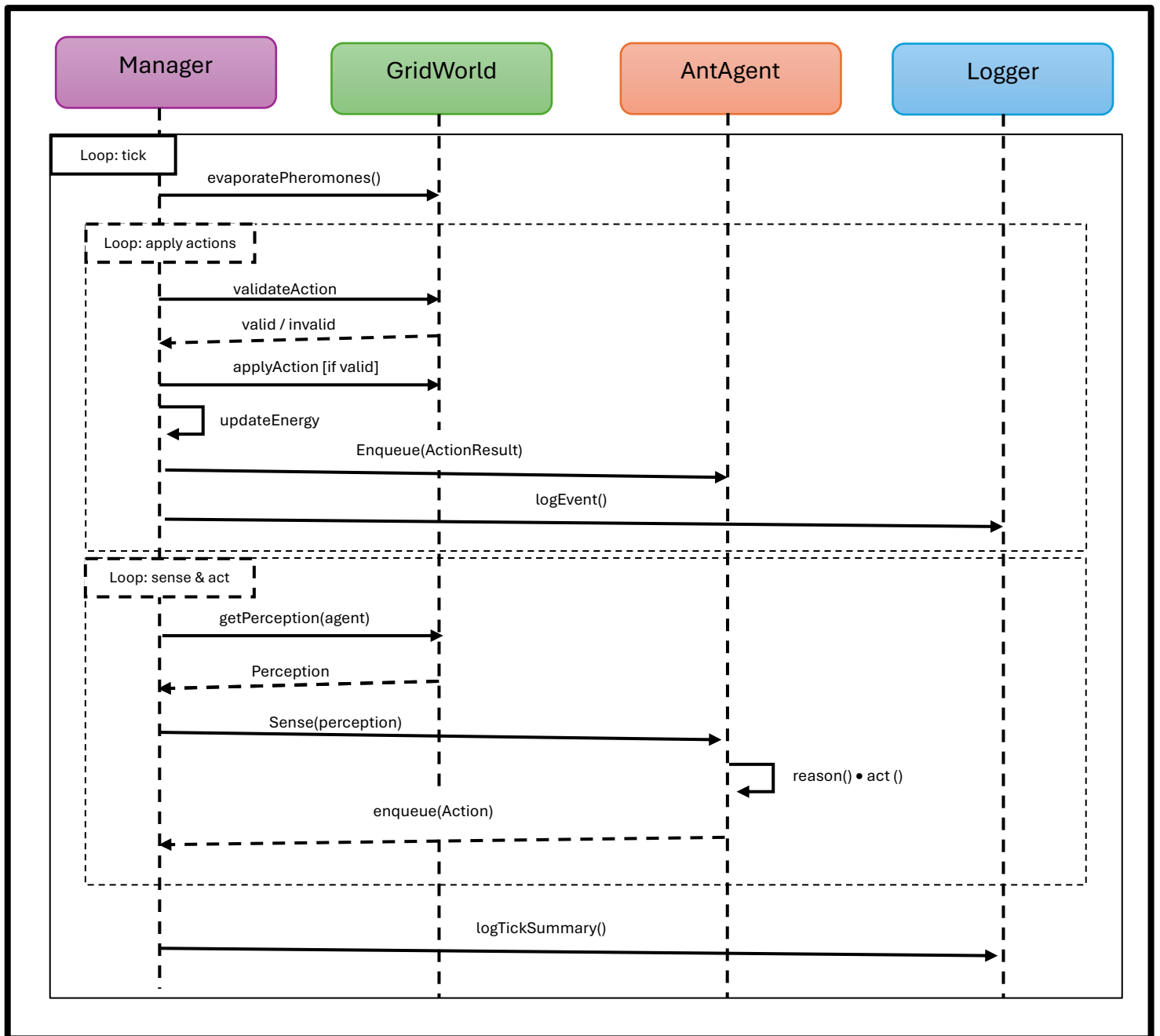
Die Modelldatei beschreibt alle relevanten Startparameter eines Experiments. Dazu gehören die Größe der Gridwelt, die Nachbarschaftsart, besondere Felder wie Nest, Nahrungsquellen und Hindernisse, die Anzahl und Eigenschaften der Ameisen sowie Parameter für Energieverbrauch, Pheromone und Simulationsdauer.

Alle nicht explizit angegebenen Felder besitzen die Standardkapazität und enthalten zu Beginn keine Items. Besondere Felder werden im Abschnitt `cells` definiert. Dazu gehören das Nest, die Nahrungsquellen und Hindernisse (dargestellt durch eine Kapazität von 0). Das Auffüllen der Energie erfolgt implizit durch den Manager beim Betreten von Nest oder Nahrung, weshalb dafür keine separaten Items im Grid platziert werden müssen.

Um die Fähigkeit für "Warmstarts" (Experiment 2) zu demonstrieren, enthält dieses Beispiel-Grid bei der Koordinate (3, 6) bereits ein Pheromon-Item. Konform zu den Vorgaben werden Pheromone als Ganzzahlen (int) modelliert. Die Verdunstung erfolgt getaktet durch das Abziehen eines festen Wertes (`evaporation_amount: 1`).

Der Parameter `seed` sorgt dafür, dass zufällige Entscheidungen deterministisch und reproduzierbar sind. Dadurch können Simulationen mit denselben Startbedingungen wiederholt und miteinander verglichen werden. Für unterschiedliche Experimente werden separate JSON-Dateien verwendet, zum Beispiel mit variierender Ameisenanzahl (5, 15 oder 30)

6. Sequenzdiagramm



7. Logging-Konzept

Das Logging erfolgt über einen eigenen SimulationLogger. Pro Simulation werden zwei Ebenen von Daten erzeugt: ein Tick-Log und ein Event-Log.

1. Tick-Log (CSV):

- tick
- anzahl_lebender_ameisen
- anzahl_suchender_ameisen
- anzahl_tragender_ameisen
- gesammelte_nahrung_im_nest
- restnahrung_pro_quelle
- durchschnittsenergie
- erfolgreiche_aktionen
- fehlgeschlagene_aktionen
- abgelegte_pheromonmenge
- aktive_pheromonfelder

2. Event-Log (JSONL oder JSON):

- Nahrungsquelle gefunden
- Nahrung aufgenommen
- Nahrung ins Nest gebracht
- Ameise gestorben
- Aktion fehlgeschlagen
- Pheromonspur verstärkt / verdunstet
- Bewegung durch Hindernis, Gridgrenze oder volle Zelle blockiert

Zusätzlich werden kompakte Statusinformationen auf der Konsole ausgegeben.

Auswertung:

Die Logs werden nach der Simulation genutzt, um die Simulationsläufe innerhalb eines Experiments zu vergleichen. Aus dem Tick-Log werden Zeitreihen erstellt, zum Beispiel zur gesammelten Nahrung im Nest, zur Anzahl lebender Ameisen, zum Verhältnis von suchenden und tragenden Ameisen sowie zur Entwicklung der Pheromonfelder. Aus dem Event-Log werden wichtige Zeitpunkte bestimmt, etwa der erste Fund einer Nahrungsquelle, die erste Lieferung ins Nest oder der Tod von Ameisen.

Für den Vergleich werden Kennzahlen wie gelieferte Nahrung, Nahrung pro Tick, Zeitpunkt der ersten Lieferung, Überlebensrate, Anteil fehlgeschlagener Aktionen und aktive Pheromonfelder betrachtet. Optional wird zusätzlich ein Gesamt-Score berechnet:

```
score =  
100 * food_delivered_ratio  
+ 20 * (1 - first_delivery_tick / max_ticks)  
+ 15 * survival_rate  
- 10 * failed_action_ratio
```

Dabei gilt:

- $\text{food_delivered_ratio} = \text{transportierte_Nahrung} / \text{verfügbare_Nahrung}$
- $\text{survival_rate} = \text{lebende_Ameisen} / \text{initiale_Ameisen}$
- $\text{failed_action_ratio} = \text{fehlgeschlagene_Aktionen} / \text{alle_Aktionen}$

Der Score dient nur als zusätzliche Vergleichsgröße. Die eigentliche Interpretation erfolgt anhand der einzelnen Kennzahlen und Zeitreihen.

8. Geplante Experimente

Experiment 1: Nahrungsversorgung und Populationsgröße

Ziel ist zu zeigen, dass die Ameisen Nahrung finden, stabile Pheromonpfade aufbauen und Nahrung in das Nest transportieren. Dazu werden Simulationen mit unterschiedlich großen Ameisenpopulationen durchgeführt.

Simulationsreihe

- E1-A: 5 Ameisen, Energie 120
- E1-B: 15 Ameisen, Energie 120
- E1-C: 30 Ameisen, Energie 120

Erwartung:

- E1-A: Nahrung wird gefunden, aber nur langsam.
- E1-B: stabile Pfade und gute Ausbeutung.
- E1-C: schneller Fund, aber nicht proportional bessere Gesamtleistung.

Parameter:

- Grid: 18x12
- Nest: (2,6)
- Nahrung F1: (14,2), Menge 40
- Nahrung F2: (15,9), Menge 60
- Ticks: 2000
- Evaporation: 0.02
- Exploration: 0.15

Experiment 2: Warmstart mit unterbrochener Pheromonspur

Ziel ist zu prüfen, ob eine bereits vorhandene, aber unterbrochene Pheromonspur durch Exploration wiederhergestellt werden kann.

Ausgangslage:

- Es existiert bereits eine Pheromonspur vom Nest zur Nahrung.
- In der Spur wird ein Segment künstlich entfernt.

Simulationsreihe

- E2-A: Lücke von 0 Feldern
- E2-B: Lücke von 3 Feldern
- E2-C: Lücke von 6 Feldern

Erwartung:

- E2-A: stabile Spur, schnelle Lieferung.
- E2-B: die Lücke wird meist wieder geschlossen.
- E2-C: Reparatur dauert länger oder scheitert teilweise.

Parameter:

- Grid: 20x20
- Nest: (19,19)
- Nahrung: (19,1), Menge 80
- Ameisen: 20
- Energie: 140
- Ticks: 1500
- Exploration: 0.20
- Evaporation: 0.02

Experiment 3: Skalierbarkeit

Ziel ist zu untersuchen, wie sich größere Entfernungen und schnellere Pheromonverdunstung auswirken.

Simulationsreihe

- E3-A: kleines Grid, kurze Distanz, Evaporation 0.02
- E3-B: großes Grid, lange Distanz, Evaporation 0.02
- E3-C: großes Grid, lange Distanz, Evaporation 0.08

Erwartung:

- E3-A: stabiler Pfad.
- E3-B: später Fund, aber grundsätzlich möglich.
- E3-C: instabile Spuren und geringere Ausbeute.

Parameter:

- Ameisen: 25
- Energie: 180
- Ticks: 3000
- Food-Menge: 100
- Exploration: 0.15

Konkrete Koordinaten:

- E3-A: Grid 20x12, Nest (1,1), Nahrung (16,10)
- E3-B: Grid 44x20, Nest (1,1), Nahrung (40,18)
- E3-C: Grid 44x20, Nest (1,1), Nahrung (40,18)